Column Transformer

Contents

1. Overview . . . 2
2. Different columns need different transformations . . . 3
3. The COVID toy data . . . 3
   - 3.1 What each column needs . . . 4
   - 3.2 Split before transforming . . . 5
4. The hard way: one transformer at a time . . . 5
   - 4.1 Fever: fill the missing values . . . 6
   - 4.2 Cough: ordinal encoding . . . 6
   - 4.3 Gender and city: one-hot encoding . . . 7
   - 4.4 Age and the final join . . . 7
5. The easy way: ColumnTransformer . . . 8
   - 5.1 The transformers list . . . 8
   - 5.2 The remainder parameter . . . 9
   - 5.3 Building and using the column transformer . . . 9
6. Reading the output . . . 11
   - 6.1 Names of the output columns . . . 11
   - 6.2 A DataFrame instead of an array . . . 11
7. Summary . . . 13
8. Sources . . . 13
9. Key terms . . . 13

Where this fits: Pipeline map step 5 (Engineer features). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	**5 Features**	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Encoding categorical data.
- **Used here, taught in full later:** Simple imputation (mean, median, mode, constant).
- **Leads to:** ML pipelines; Function transformer; Binning and binarization.

1. Overview

Key point: A **column transformer** (G-415) sends each column to its own transformer and joins all the outputs into one array, in a single step.

Real datasets mix feature types. A **feature** (G-772) is an input variable, one column of the data table. One feature has **missing values** (G-1234), another is ordinal, a third is nominal, and a fourth needs nothing at all. Each one needs a different transformation.

Figure 1 shows the whole topic. Every column goes to the transformer that fixes its problem, and scikit-learn's `ColumnTransformer` (G-417) runs all of them at once and joins their outputs into one array of numbers.

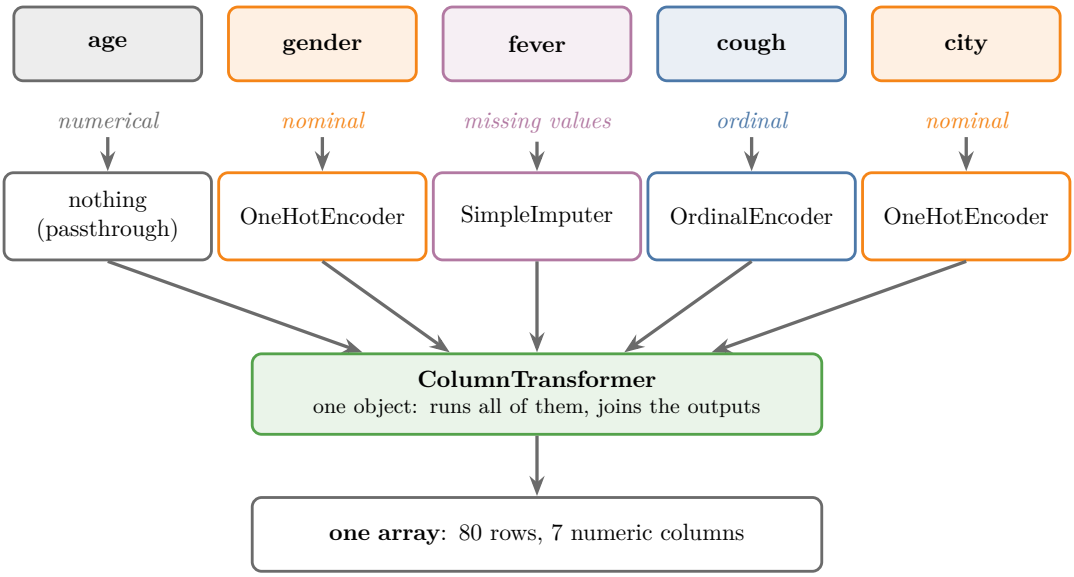


Figure 1: Each column of the COVID data and the transformer it needs; ColumnTransformer runs them all and joins the outputs

2. Different columns need different transformations

> **Key point:** When columns need different transformations, doing them one by one means many separate arrays that we must glue back together.

Take customer data with four **features** (G-772):

- **age:** numerical, but some values are **missing values** (G-1234). Age needs **imputation** (G-927) to fill them.
- **city:** nominal categories. City needs one-hot encoding.
- **gender:** nominal categories. Gender also needs one-hot encoding.
- **review:** ordinal categories, such as good and excellent. Review needs ordinal encoding.

If we apply each transformation separately, we get three separate NumPy arrays: one from imputing age, one from one-hot encoding city and gender, and one from ordinal encoding review. We then have to join the three into one big array before we can train a model.

Gluing arrays by hand is a lot of manual work, and the work grows with every feature. Think of a kitchen where each cook prepares one ingredient: a column transformer is the head chef who hands each ingredient to the right cook and plates the results together. A **column transformer** (G-415) is a scikit-learn class that does the whole job in one go: it takes the DataFrame (the pandas table of rows and columns; see loading and cleaning the data) and returns one finished array (a NumPy table of numbers).

3. The COVID toy data

> **Key point:** 100 made-up patients; fever has **missing values** (G-1234), cough is ordinal, gender and city are nominal, and age is ready as it is.

The data is a small made-up dataset of 100 patients. Each patient is one **observation** (G-1374) (one record, one row of the table). There are six columns: the **features** (G-772) age, gender, fever (body temperature in degrees Fahrenheit at the doctor's visit), cough and city, and the **target** (G-1949) `has_covid` (Yes or No), the output we would predict. The dataset is a toy for practising the technique, not real medical data.

age	gender	fever	cough	city	has_covid
60	Male	103.0	Mild	Kolkata	No
27	Male	100.0	Mild	Delhi	Yes
42	Male	101.0	Mild	Delhi	No

A few quick counts describe the columns (Figure 2):

- **cough:** 62 patients have a Mild cough and 38 a Strong one.
- **city:** four cities: Kolkata (32), Bangalore (30), Delhi (22) and Mumbai (16).
- **fever:** 10 of the 100 values are missing.

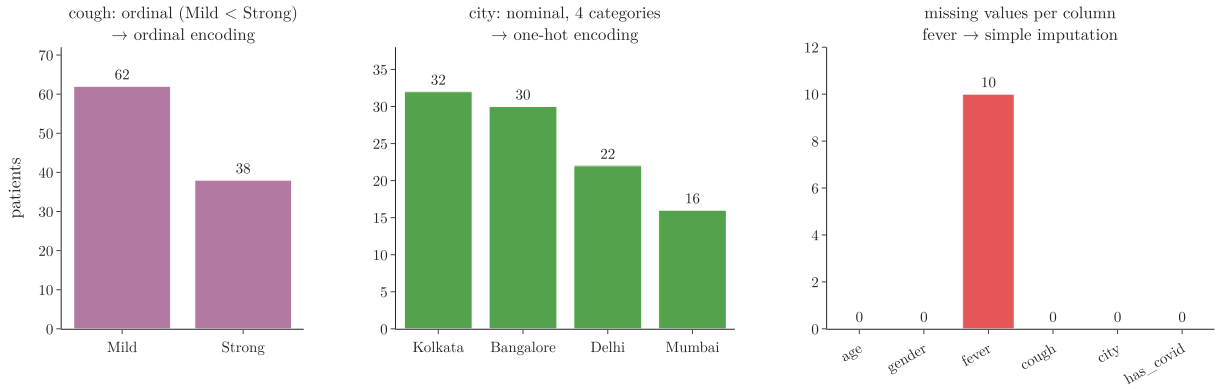


Figure 2: The COVID toy data: the two cough categories, the four cities, and the missing values per column. Each panel names the transformer that fixes it.

Python: Loading the data and counting values.

```
import pandas as pd

df = pd.read_csv("data/covid_toy.csv")
# how many rows hold each category
df["cough"].value_counts()
df["city"].value_counts()
# missing values per column: fever has 10
df.isnull().sum()
```

3.1 What each column needs

Key point: Four **features** (G-772) need a transformation; age needs none.

Column	Type	Problem	Transformation
age	numerical	none	none
fever	numerical	10 missing values	simple imputation (`SimpleImputer`)
cough	ordinal: Mild < Strong	text	ordinal encoding (`OrdinalEncoder`)
gender	nominal	text	one-hot encoding (`OneHotEncoder`)
city	nominal	text	one-hot encoding (`OneHotEncoder`)

The **target** (G-1949) `has_covid` is categorical too, so it would need label encoding. We leave it out here: the focus is the features.

3.2 Split before transforming

> **Key point:** As always, we split first; every transformer learns from the training set only.

We split the data into a training set (80 rows) and a test set (20 rows) before any transformation, so that no number learned by a transformer (such as the mean fever) comes from the test rows; see split before scaling. The **features** (G-772) are every column except `has_covid`.

> **Python:** Splitting the data.

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df.drop(columns=["has_covid"]),  # inputs
    df["has_covid"],                 # target
    test_size=0.2, random_state=0)
# X_train: (80, 5), X_test: (20, 5)
```

4. The hard way: one transformer at a time

> **Key point:** Without a **column transformer** (G-415), we transform each column separately, get one array per step, and glue the arrays together with `np.concatenate`.

Figure 3 shows the plan. Four separate steps each give an array of 80 rows, and a fifth step joins them into one array of 80 rows and 7 columns.

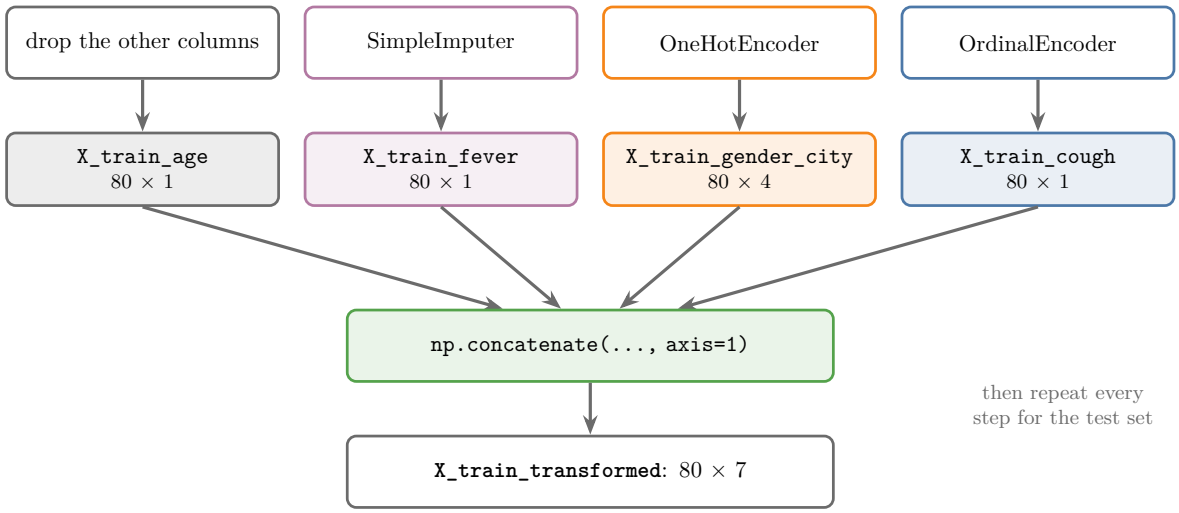


Figure 3: Transforming by hand: one array per step, joined at the end

4.1 Fever: fill the missing values

> **Key point:** `SimpleImputer` replaces each missing fever with the mean fever of the training set.

`SimpleImputer` (G-1809), with its default setting, fills every **missing value** (G-1234) with the column's mean. This is **simple imputation**: each gap gets one value computed from the known values of the column, such as their mean or median (mean and median imputation, taught in full later). We pick only the `fever` column, fit the imputer on the training set and transform both sets.

> **Python:** Imputing fever.

```
from sklearn.impute import SimpleImputer

si = SimpleImputer()
# double brackets keep a table of one column
X_train_fever = si.fit_transform(X_train[["fever"]])
X_test_fever = si.transform(X_test[["fever"]])
# X_train_fever: (80, 1), no missing values left
```

`fit_transform` does `fit` and `transform` in one call. We use it on the training set only; the test set gets just `transform`.

The number the imputer learns, in two lines. Of the 80 training fevers, 9 are missing and 71 are known; the known ones add up to 7,165:

$$\frac{7165}{71} = 100.92$$

Every missing fever, in the training and the test set, becomes 100.92 (the Note rounds it to 100.9 in Figure 5).

4.2 Cough: ordinal encoding

> **Key point:** Mild becomes 0 and Strong becomes 1.

Cough has an order, Mild < Strong, so we use `OrdinalEncoder` and give it that order.

> **Python:** Ordinal encoding cough.

```
from sklearn.preprocessing import OrdinalEncoder

oe = OrdinalEncoder(categories=[["Mild", "Strong"]])
X_train_cough = oe.fit_transform(X_train[["cough"]])
X_test_cough = oe.transform(X_test[["cough"]])
# X_train_cough: (80, 1)
```

4.3 Gender and city: one-hot encoding

> **Key point:** Gender (2 categories) and city (4 categories) become $1 + 3 = 4$ columns once the first category of each is dropped.

Both columns are nominal, so one `OneHotEncoder` handles the two together. With `drop="first"` it removes one column per **feature** (G-772) to avoid **multicollinearity** (one input column being exactly predictable from the others; see **multicollinearity** (G-1273)).

The column count works out as follows:

- **gender:** 2 categories, minus the first, gives 1 column.
- **city:** 4 categories, minus the first, gives 3 columns.
- **Total:** $1 + 3 = 4$ columns.

Python: One-hot encoding gender and city.

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(drop="first", sparse_output=False)
X_train_gender_city = ohe.fit_transform(
    X_train[["gender", "city"]])
X_test_gender_city = ohe.transform(
    X_test[["gender", "city"]])
# X_train_gender_city: (80, 4)
```

`sparse_output=False` returns an ordinary array instead of a sparse matrix.

4.4 Age and the final join

> **Key point:** Age is taken out unchanged, then `np.concatenate` joins the four arrays side by side.

Age needs no transformation, but it still has to become an array so it can be joined to the others. Then **`np.concatenate`** (G-1355) with `axis=1` puts the arrays next to each other, column after column. Age, fever, the one-hot block and cough give:

$$1 + 1 + 4 + 1 = 7 \text{ columns}$$

Python: Extracting age and joining everything.

```
import numpy as np

X_train_age = X_train[["age"]].values
X_test_age = X_test[["age"]].values

# axis=1: join side by side (column-wise)
X_train_transformed = np.concatenate(
    (X_train_age, X_train_fever,
```

```
     X_train_gender_city, X_train_cough), axis=1)
X_test_transformed = np.concatenate(
    (X_test_age, X_test_fever,
     X_test_gender_city, X_test_cough), axis=1)
# (80, 7) and (20, 7)
```

The hand-made way works, but it is hectic for only four **features** (G-772). With 50 columns, it would take a very long time, and every step has to be written twice: once for the training set and once for the test set.

5. The easy way: ColumnTransformer

> **Key point:** We list every transformation once, as (name, transformer, columns), and `ColumnTransformer` does all the steps of Section 4 in one call.

`ColumnTransformer` lives in `sklearn.compose`. When we create it, we pass two things: the list of transformers and what to do with the remaining columns.

5.1 The transformers list

> **Key point:** Each transformation is a tuple of three items: a name, a transformer object, and a list of column names.

The parameter **`transformers`** is a list of tuples, one tuple per transformation. Figure 4 shows the three items inside one tuple.

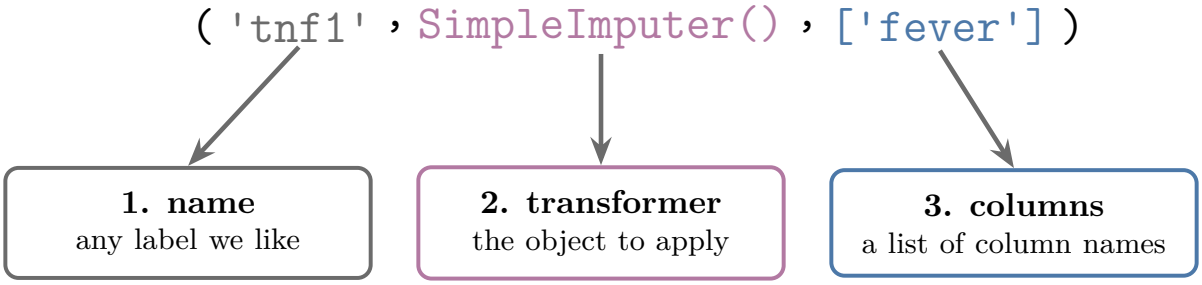


Figure 4: The three items of one tuple in the transformers list

1. **Name:** any label we choose, such as `"tnf1"`.
2. **Transformer:** the object to apply, such as `SimpleImputer()`.
3. **Columns:** a list of the columns to apply it to, such as `["fever"]`.

Our data needs three tuples: one for the imputer, one for the ordinal encoder, and one for the one-hot encoder.

5.2 The remainder parameter

> **Key point:** `remainder` decides what happens to columns that no transformer touches: `"drop"` removes them, `"passthrough"` keeps them unchanged.

Sometimes we do not transform every column. The parameter **`remainder`** (G-1669) says what to do with the rest:

- **`"drop"`:** remove them from the output. Dropping is the default.
- **`"passthrough"`** (G-1460): keep them as they are.

Here age has no transformer. We want to keep it, so we use `remainder="passthrough"`. With `"drop"`, age would disappear and the output would have 6 columns.

5.3 Building and using the column transformer

> **Key point:** `fit_transform` on the training set and `transform` on the test set give the same 80×7 and 20×7 arrays as the hard way.

> **Python:** The whole of Section 4 in one object.

```
from sklearn.compose import ColumnTransformer

transformer = ColumnTransformer(transformers=[
    ("tnf1", SimpleImputer(), ["fever"]),
    ("tnf2", OrdinalEncoder(
        categories=[["Mild", "Strong"]]), ["cough"]),
    ("tnf3", OneHotEncoder(
        sparse_output=False, drop="first"),
     ["gender", "city"]),
], remainder="passthrough")

# learn from the training set and transform it
transformer.fit_transform(X_train).shape    # (80, 7)
# apply to the test set
transformer.transform(X_test).shape         # (20, 7)
```

> The column names in each tuple must match the DataFrame exactly, including case: `["Fever"]` for a column called `fever` stops with an error saying the column does not exist.

Figure 5 shows what happens inside, using three training rows, one of them with a missing fever. The columns are routed to their transformers, each transformer works only on its own columns, and the outputs are stitched side by side. That missing fever is filled with 100.9, the mean fever of the training set.

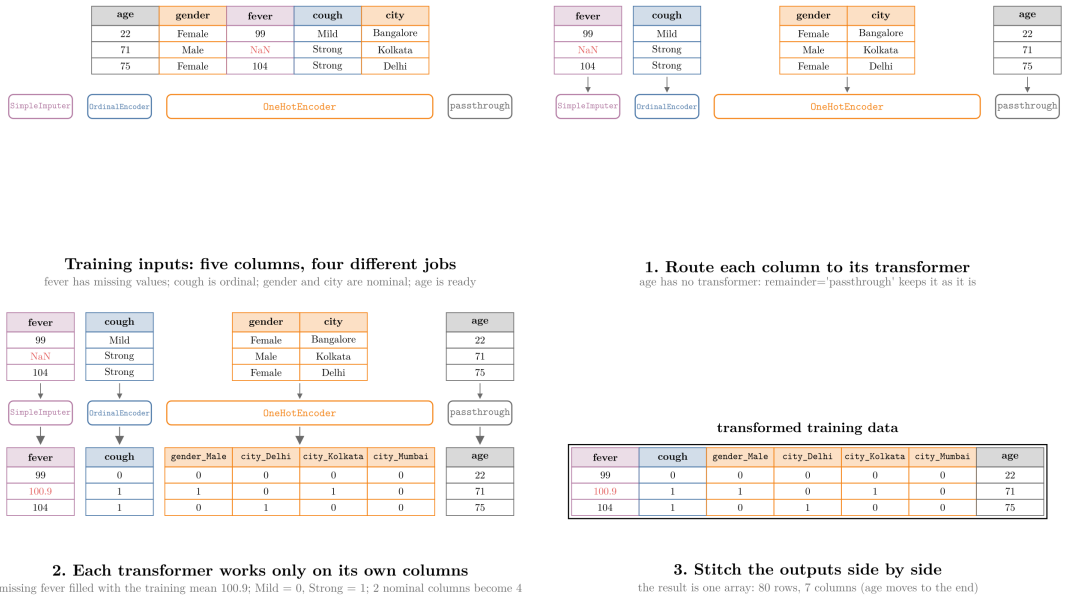


Figure 5: ColumnTransformer step by step: route, transform, stitch

On the real output, the first three training patients look like this before the transformation:

Patient	age	gender	fever	cough	city
1	22	Female	99.0	Mild	Bangalore
2	56	Female	104.0	Strong	Bangalore
3	31	Female	98.0	Mild	Kolkata

After it, they are three rows of 7 numbers. The table below is turned on its side to fit: each line is one output column, and each number column is one patient.

Output column	Patient 1	Patient 2	Patient 3
fever	99	104	98
cough	0	1	0
`gender_Male`	0	0	0
`city_Delhi`	0	0	0
`city_Kolkata`	0	0	1
`city_Mumbai`	0	0	0
age	22	56	31

Bangalore is the first city alphabetically, so it was the dropped category: a Bangalore row has 0 in all three city columns.

The **column transformer** (G-415) combines naturally with another scikit-learn tool, the **pipeline** (G-1499; a chain of steps run in order, what a pipeline is). Used together, they make preprocessing much simpler.

> **Extra:** The output columns come in the order of the `transformers` list, and the `remainder` columns go last. So the column transformer puts age at the end, while our hand-made array had it first. Both arrays hold the same 7 columns, only in a different order.

6. Reading the output

> **Key point:** `get_feature_names_out` gives the name of every output column, and `set_output(transform="pandas")` returns a DataFrame with those names instead of a bare array.

> **Extra:** Section 6 goes beyond the basics: it uses newer scikit-learn tools that make the output easier to read.

The result of `fit_transform` is a NumPy array without column names, so it is easy to forget which column is which. Two tools fix this.

6.1 Names of the output columns

> **Key point:** Each output name is the transformer's name, two underscores, then the column name.

`get_feature_names_out` (G-846) lists the output columns in order. Each name starts with the transformer's name and two underscores; columns kept by `remainder` start with `remainder__`.

> **Python:** Listing the output column names.

```
transformer.get_feature_names_out()
# ['tnf1__fever', 'tnf2__cough',
#  'tnf3__gender_Male', 'tnf3__city_Delhi',
#  'tnf3__city_Kolkata', 'tnf3__city_Mumbai',
#  'remainder__age']
```

6.2 A DataFrame instead of an array

> **Key point:** After `set_output(transform="pandas")`, the **column transformer** (G-415) returns a DataFrame.

Calling **`set_output(transform="pandas")`** (G-1780) once makes every later `fit_transform` and `transform` return a pandas DataFrame, with the names above as its columns. Figure 6 shows the result for the first three training patients; the header colours mark which transformer made each column.

First three training patients after set_output(transform="pandas")

tnf1__ fever	tnf2__ cough	tnf3__ gender_Male	tnf3__ city_Delhi	tnf3__ city_Kolkata	tnf3__ city_Mumbai	remainder__ age
99	0	0	0	0	0	22
104	1	0	0	0	0	56
98	0	0	0	1	0	31

Figure 6: The first three training patients after set_output(transform="pandas"). Red: the imputer (tnf1), purple: the ordinal encoder (tnf2), green: the one-hot encoder (tnf3), grey: the remainder.

Python: Getting a DataFrame back.

```
transformer.set_output(transform="pandas")
transformer.fit_transform(X_train).head()
# a DataFrame with columns tnf1__fever ... remainder__age
```

Passing `verbose_feature_names_out=False` to `ColumnTransformer` drops the prefixes, giving plain names such as `fever` and `city_Delhi`.

Extra: The fitted transformers are kept inside the column transformer, under their names. For example, `transformer.named_transformers_["tnf1"].statistics_` gives `[100.92]`, the training mean used to fill the missing fevers.

7. Summary

	One transformer at a time	ColumnTransformer
Objects	one per transformation	one
Choosing columns	by hand, for each step	in the `transformers` list
Untouched columns	extracted by hand	`remainder="passthrough"`
Joining the outputs	`np.concatenate` by hand	automatic
Test set	every step written again	one `transform` call
Result for this data	80×7 (age first)	80×7 (age last)

- Different columns often need different transformations: **imputation** (G-927), ordinal encoding, one-hot encoding, or none, because each column has its own problem (gaps, ordered text, unordered text).
- Done by hand, each transformation gives its own array, and we must join them with `np.concatenate`, for both sets, so the work grows with every **feature** (G-772) and every step is written twice.
- `ColumnTransformer` takes a list of (name, transformer, columns) tuples and does every step in one call, so each transformation is written once and the test set needs only one `transform`.
- `remainder="drop"` (default) removes untouched columns; `remainder="passthrough"` keeps them, at the end. Forgetting it silently loses a ready column such as age (6 columns instead of 7).
- Split first; `fit_transform` on the training set, `transform` on the test set, so every transformer learns from the training set only (the missing fevers of both sets get the training mean, 100.92).
- `get_feature_names_out` and `set_output(transform="pandas")` tell us which output column is which, because the plain output is an array without column names.
- So one **column transformer** (G-415) replaces all the hand-made steps: each column goes to its own transformer, and the outputs come back as one array.

8. Sources

Built from

- CampusX, "Column Transformer in Machine Learning | How to use ColumnTransformer in Sklearn", YouTube, https://www.youtube.com/watch?v=5TVj6iEBR4I

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Column transformer (G-415)	A scikit-learn class that applies different transformations to different columns at once and joins the results.
ColumnTransformer (G-417)	The scikit-learn class (in `sklearn.compose`) that implements the column transformer.
SimpleImputer (G-1809)	scikit-learn's class that fills missing values, by default with the column's mean.
np.concatenate (G-1355)	NumPy function that joins arrays; with `axis=1` it puts them side by side.
remainder (G-1669)	The `ColumnTransformer` parameter for untouched columns: `"drop"` (default) or `"passthrough"`.
passthrough (ColumnTransformer) (G-1460)	The `remainder` option that keeps untouched columns unchanged.
set_output (G-1780)	Method that, called as `set_output(transform="pandas")`, makes a scikit-learn transformer return a pandas DataFrame with column names instead of a bare NumPy array.
fit_transform (G-785)	Learns the numbers from the data and applies them in one call (fit, then transform); used on the training set only.
transformers (G-2008)	The `ColumnTransformer` parameter that says which transformer to apply to which columns: a list of (name, transformer, columns) tuples.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Missing value (G-1234)	An empty entry, shown by pandas as `NaN`.
Imputation (G-927)	Filling in missing values, for example with the mean, median or mode.
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Multicollinearity (G-1273)	A relationship between input columns, so that one can be calculated (or largely predicted) from the others; it makes a linear model's coefficients unstable and hard to interpret.

Term	Meaning
Pipeline (G-1499)	One object that chains preprocessing steps and a model, each step's output feeding the next; every input, including new ones after deployment, then gets exactly the same preprocessing.
get_feature_names_out (G-846)	`OneHotEncoder` method that returns the names of the new columns.